

Computer Hardware

Inside the Processor

Ksawery Możdżyński

Agenda For Today

1. Addressing Modes

Direct and indirect addressing, arrays, and immediate values

2. How the CPU Talks to Memory

MAR, MBR, the bus, micro-operations, instruction steps, and the von Neumann bottleneck

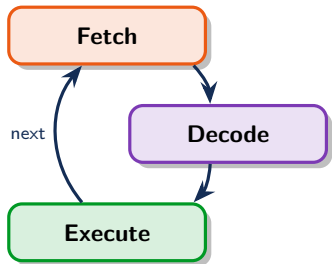
3. The Control Unit

How control signals are generated: hardwired and microprogrammed control

4. RISC and CISC

Operand sources, instruction length, a comparison exercise, and RISC/CISC today

Where we stopped on Day 1



PC – points to the next instruction

IR – holds the instruction being executed

AC – holds the working value

ALU calculates, **CU** directs

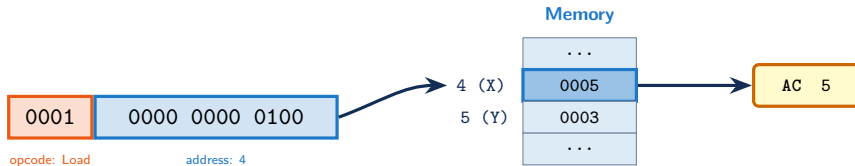
MARIE instructions you know

Load, Store, Add, Subt, Input, Output, Jump, Skipcond, Halt

Day 1: **what** the processor does. Today: **how** it does it inside.

How does an instruction point to its data?

In memory, Load X (with X at address 4) is stored as 1004: opcode 1 and address 004. The address is **part of the instruction**.



The assembler writes the address into the instruction when it translates the program. After that, the address **never changes**.

This way of pointing to the data is called **direct addressing**.

A problem: adding up an array

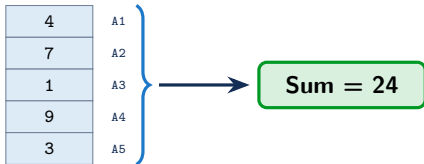
Five numbers are stored in neighbouring cells A1...A5. We can add them one by one.

A direct solution:

```
0  Load A1
1  Add  A2
2  Add  A3
3  Add  A4
4  Add  A5
5  Store Sum
```

The program works, but it contains one instruction for every array element.

Array values



The result is $4 + 7 + 1 + 9 + 3 = 24$.

The problem with repeating instructions

The direct solution works, but it does not scale. What if the array has 100 or 1000 elements?

A first attempt at a loop:

```
0  Loop, Load Sum
1      Add A1    / always A1!
2      Store Sum
3      Jump Loop
```

The loop repeats, but it keeps adding the value from **A1**. The address in Add A1 is fixed inside the instruction.

To walk through an array, we need an address that can **change while the program runs**.

The next idea

Keep the address in a memory cell, where the program can change it.

The instruction decides what a value means

Memory

18	25
	...
25	4
26	7
27	1

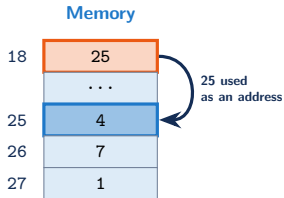
Load 18 (direct)

Takes the value from cell 18 and treats it as **data**.

Result: AC = 25

The memory does not know what 25 means. **The instruction decides:** data or address.

The instruction decides what a value means



Load 18 (direct)

Takes the value from cell 18 and treats it as **data**.

Result: AC = 25

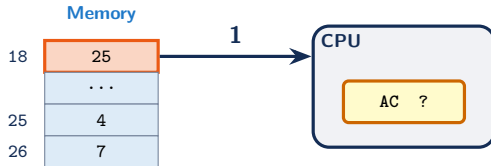
LoadI 18 (indirect)

Takes the value from cell 18 and treats it as an **address**: goes to cell 25 and takes the value from there.

Result: AC = 4

The memory does not know what 25 means. **The instruction decides:** data or address.

LoadI 18 step by step: two memory reads



Instruction being executed

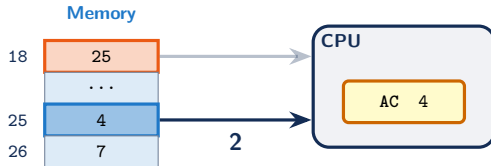
LoadI 18

Step 1

The CPU reads cell 18 and gets **25**. LoadI treats it as an **address**.

More memory reads mean more work for the processor. We will measure this later today.

LoadI 18 step by step: two memory reads



Instruction being executed

LoadI 18

Step 1

The CPU reads cell 18 and gets **25**. LoadI treats it as an **address**.

Step 2

The CPU reads cell **25** and gets **4**. The value goes into the AC.

State: AC = 4

More memory reads mean more work for the processor. We will measure this later today.

Indirect instructions in MARIE

Instruction	Effect	Opcode	In words
LoadI X	$AC \leftarrow M[M[X]]$	D	bring the value from the address stored in X
StoreI X	$M[M[X]] \leftarrow AC$	E	write the AC to the address stored in X
AddI X	$AC \leftarrow AC + M[M[X]]$	B	add the value from the address stored in X
JumpI X	$PC \leftarrow M[X]$	C	jump to the address stored in X

The letter **I** stands for **indirect**.

$M[M[X]]$: first read cell X, then use what you found as an address.

A program that walks through an array



```
0  Loop, LoadI Addr    / read element
1      Skipcond 800    / > 0? skip Halt
2      Halt
3      Output
4      Load Addr
5      Add One
6      Store Addr      / Addr + 1
7      Jump Loop
8  One,  DEC 1
9  Addr, ADR Arr       / address of Arr
10 Arr,  DEC 4
11      DEC 7
12      DEC 1
13      DEC 0          / end of array
```

ADR Arr

The assembler puts the **address** of Arr into Addr.

LoadI Addr

reads the value from the address stored in Addr.

Load, Add, Store

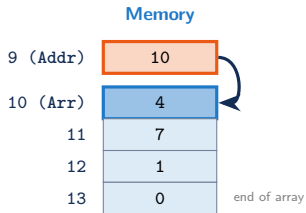
increase the address in Addr by 1, so the next pass reads the next cell.

Skipcond 800

The array ends with **0**: then Halt is not skipped and the program stops. (Elements must be greater than 0.)

The program prints **4, 7, 1**: every element of the array, with one LoadI instruction.

Walking through the array, pass by pass

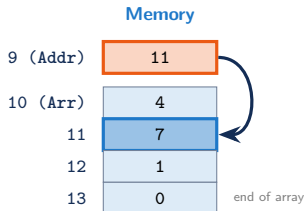


Pass	Value in Addr	LoadI	Addr reads	Printed
1	10		4	4

The assembler placed `Arr` at address 10, so `ADR Arr` put 10 into `Addr`.

The instruction `LoadI Addr` never changes. Only the **number stored in Addr** changes.

Walking through the array, pass by pass

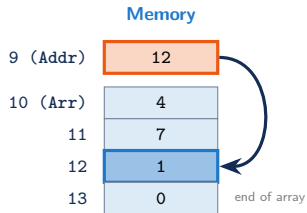


Pass	Value in Addr	LoadI	Addr reads	Printed
1	10		4	4
2	11		7	7

The assembler placed `Arr` at address 10, so `ADR Arr` put 10 into `Addr`.

The instruction `LoadI Addr` never changes. Only the **number stored in Addr** changes.

Walking through the array, pass by pass

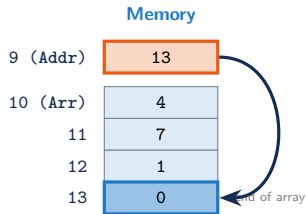


Pass	Value in Addr	LoadI	Addr reads	Printed
1	10		4	4
2	11		7	7
3	12		1	1

The assembler placed `Arr` at address 10, so `ADR Arr` put 10 into `Addr`.

The instruction `LoadI Addr` never changes. Only the **number stored in Addr** changes.

Walking through the array, pass by pass



Pass	Value in Addr	LoadI	Addr reads	Printed
1	10		4	4
2	11		7	7
3	12		1	1
4	13		0	– (stop)

Pass 4 reads 0: Skipcond does not skip, so Halt runs.

The assembler placed Arr at address 10, so ADR Arr put 10 into Addr.

The instruction LoadI Addr never changes. Only the **number stored in Addr** changes.

Exercise 1: sum of an array



```
Sum,   DEC 0
One,   DEC 1
Addr,  ADR Arr / address of Arr
Arr,   DEC 4
       DEC 7
       DEC 1
       DEC 9
       DEC 3
       DEC 0 / end of array
```

The data part of the program. Write the code above it.

Task

Write the program in MARIE.js.

1. Change the program from the previous slides: instead of printing every element, **add them up** and print only the sum.
2. Read the elements with LoadI Addr.
3. Test: the program should print **24**.

Hint: the loop ends the same way as in the printing program: on the value 0.

Solution: sum of an array



```
0  Loop,  LoadI Addr    / read element
1         Skipcond 800   / > 0? skip Jump
2         Jump Done
3         Add Sum        / element + Sum
4         Store Sum
5         Load Addr
6         Add One
7         Store Addr     / Addr + 1
8         Jump Loop
```



```
9  Done,  Load Sum
10         Output
11         Halt
12  Sum,   DEC 0
13  One,   DEC 1
14  Addr,  ADR Arr      / address of Arr
15  Arr,   DEC 4
16         DEC 7
17         DEC 1
18         DEC 9
19         DEC 3
20         DEC 0        / end of array
```

Idea: each pass reads the element at the address stored in Addr, adds it to Sum and increases Addr by 1, until it reads 0.

Exercise 2: the largest element



```
Max, DEC 0
One, DEC 1
Addr, ADR Arr / address of Arr
Arr, DEC 4
    DEC 7
    DEC 1
    DEC 9
    DEC 3
    DEC 0 / end of array
```

The data part of the program. Write the code above it.

Task

Write the program in MARIE.js.

1. Find the **largest** element of the array and print it.
2. Read the elements with LoadI Addr. The array ends with 0, as before.
3. Test: the program should print **9**.

Hints: start by storing the first element in Max. MARIE has no “compare” instruction: compute *element* – Max with Subt and check the sign of the result with Skipcond.

Solution: the largest element

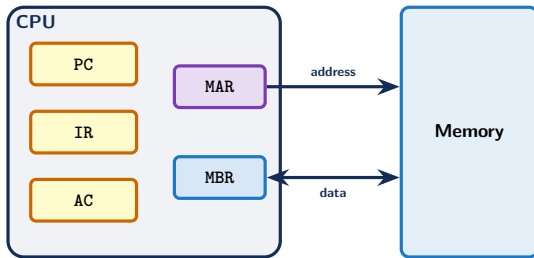
```
0      LoadI Addr
1      Store Max      / Max = first element
2  Loop, Load Addr
3      Add One
4      Store Addr      / next element
5      LoadI Addr
6      Skipcond 800    / > 0? skip Jump
7      Jump Done
8      Subt Max        / element - Max
9      Skipcond 800    / > 0? skip Jump
10     Jump Loop
11     LoadI Addr
```

No “compare” instruction: we compare by **subtracting** and checking the sign of the result.

```
12     Store Max      / new maximum
13     Jump Loop
14  Done, Load Max
15     Output
16     Halt
17  Max, DEC 0
18  One, DEC 1
19  Addr, ADR Arr      / address of Arr
20  Arr, DEC 4
21     DEC 7
22     DEC 1
23     DEC 9
24     DEC 3
25     DEC 0      / end of array
```

MAR and MBR: between the CPU and memory

The memory is a **separate part**, outside the CPU and connected to it by wires. The CPU cannot just look into a cell: it must **send an address** and **receive a value**.



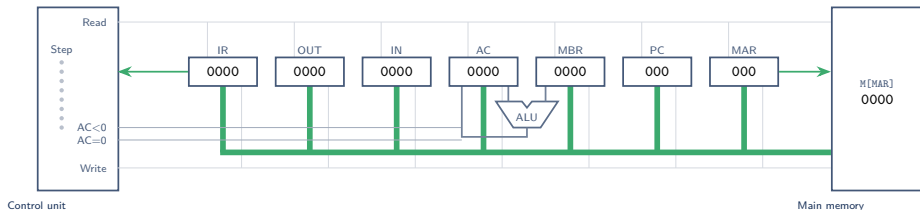
MAR (Memory Address Register) holds the **address** the CPU sends to memory.

MBR (Memory Buffer Register) holds the **value** coming from memory or going to memory.

Like a post office counter
you give the box number (MAR) and receive the parcel (MBR).

You can see both registers in MARIE.js, next to PC, IR and AC.

The bus: one road for everyone



The bus

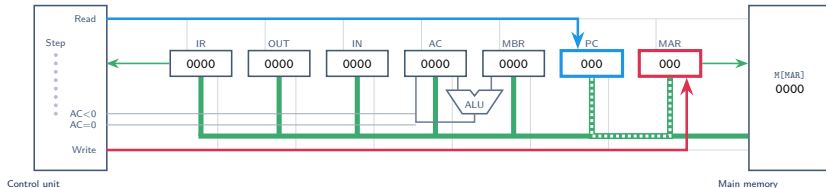
MARIE connects its registers and the memory with **one shared bus**. Values travel between them along this bus.

One value at a time

At any moment the bus can carry **only one value**. The ALU has its own direct connections to the AC and the MBR.

So moving data takes **several steps**, one after another. One step is one **micro-operation**, e.g. $MAR \leftarrow PC$. We number the steps $T_0, T_1, T_2, \dots$

Read and Write: how the control unit drives the bus



Read: puts its value on the bus

Write: stores the value from the bus

the value travelling on the bus

Read

The control unit tells one element: “put your value on the bus”. It is marked **blue**.

Write

The control unit tells one element: “store the value from the bus”. It is marked **red**.

In a micro-operation $X \leftarrow Y$: Y gets **Read**, X gets **Write**. Example above: $MAR \leftarrow PC$.

The program we will follow

Exercise 1 from Day 1



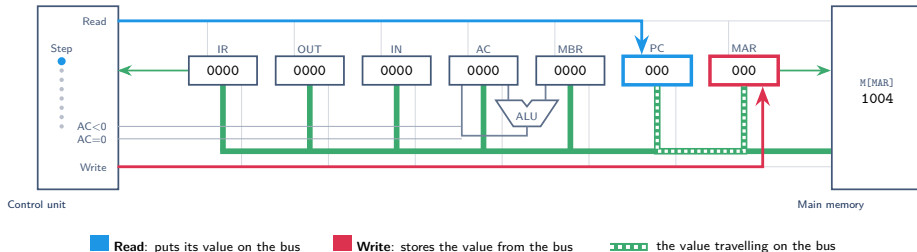
```
0 Load X
1 Add Y
2 Store Z
3 Halt
4 X, DEC 5
5 Y, DEC 3
6 Z, DEC 0
```

Memory after assembling

Address	Contents	Meaning
0	1004	Load X ← we follow this one
1	3005	Add Y
2	2006	Store Z
3	7000	Halt
4	0005	X = 5
5	0003	Y = 3
6	0000	Z = 0

We follow the first instruction, Load X, step by step. In MARIE.js you can watch the same steps with the **Micro step** button: the **Step** dots in the control unit show T0, T1, T2, ...

Fetch, step T0: $MAR \leftarrow PC$

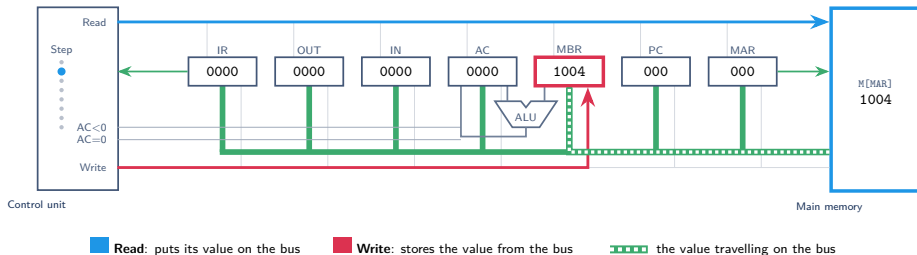


T0

The PC puts the address of the next instruction (0) on the bus, and the MAR stores it. The memory now knows **which cell** the CPU wants.

Name of this phase: **fetch**

Fetch, step T1: $MBR \leftarrow M[MAR]$

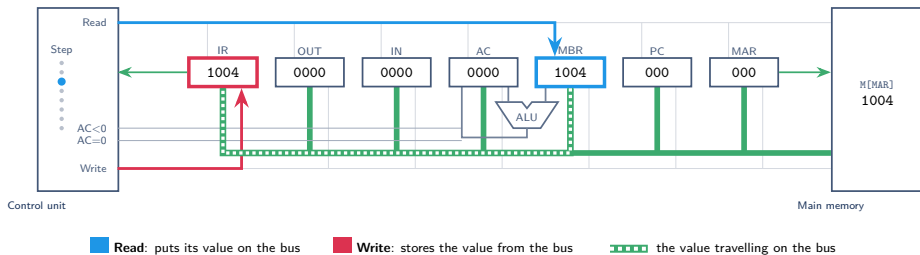


T1

The memory returns the contents of cell 0: 1004 (Load X). The instruction waits in the MBR.

Name of this phase: **fetch**

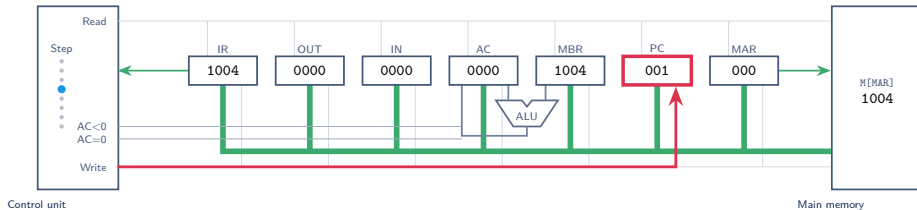
Fetch, step T2: $IR \leftarrow MBR$

**T2**

The instruction moves from the MBR into the IR.

Name of this phase: **fetch**

Fetch, step T3: $PC \leftarrow PC + 1$



Read: puts its value on the bus **Write:** stores the value from the bus **the value travelling on the bus**

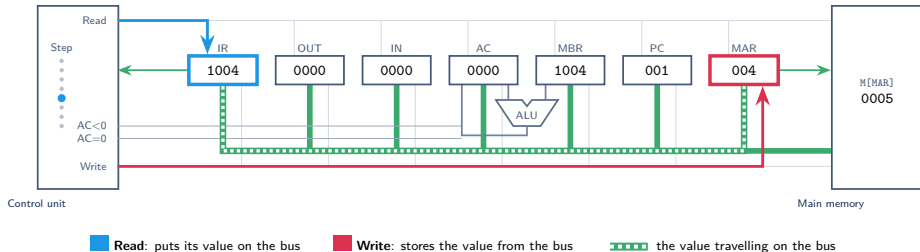
T3

The PC is increased by 1. It already points to the next instruction.

Name of this phase: **fetch**

Day 1: $IR \leftarrow M[PC]$, $PC \leftarrow PC + 1$. In reality: **4 steps**.

Execute Load X, step T4: $MAR \leftarrow IR[11-0]$

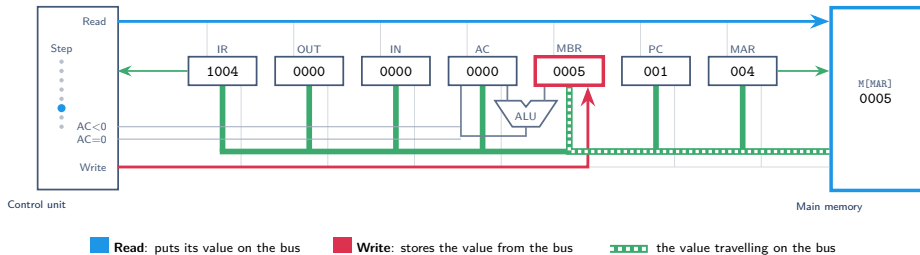


T4

The address part of the instruction (4, the address of X) goes into the MAR. IR[11-0] means bits 11 to 0 of the IR: the address field.

Name of this phase: **execute**

Execute Load X, step T5: $MBR \leftarrow M[MAR]$

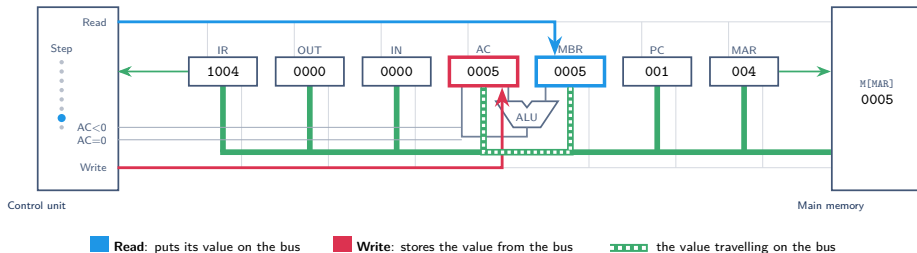


T5

The memory returns the contents of cell 4: 5. The value waits in the MBR.

Name of this phase: **execute**

Execute Load X, step T6: $AC \leftarrow MBR$

**T6**

The value moves from the MBR into the AC.

Name of this phase: **execute**

Same result as on Day 1, now step by step.

How long does one instruction take?



Step	Micro-operation	Phase
T0	MAR ← PC	fetch
T1	MBR ← M[MAR]	fetch
T2	IR ← MBR	fetch
T3	PC ← PC + 1	fetch
–	the CU decodes the opcode in IR[15–12]	decode
T4	MAR ← IR[11–0]	execute
T5	MBR ← M[MAR]	execute
T6	AC ← MBR	execute

Fetch

is the **same** for every instruction.

Decode

moves no data. The decoder works out the opcode while the PC is being increased, so in MARIE it needs no step of its own.

Execute

depends on the instruction. E.g. LoadI needs **9 steps**, because it reads the memory twice to get its value.

Exercise 3: count the control signals



```
0 Load X
1 Add Y
2 Store Z
3 Output
4 Jump Stop
5 Stop, Halt
6 X, DEC 5
7 Y, DEC 3
8 Z, DEC 0
```

Task

Press **Assemble**, turn on **Data Path** and use **Micro step**.

1. Choose one instruction: Add Y, Store Z, Output or Jump Stop.
2. Go through it step by step, **including its fetch**. Count how many times the control unit sends a **Read** signal and how many times a **Write** signal.
3. In which step is there a Write but no Read? Why?

Hint: in a micro-operation $X \leftarrow Y$, Y is marked **blue** (Read: it puts its value on the bus) and X is marked **red** (Write: it stores the value). The Step dots show the current step; Micro step also stops once for the decode.

Solution: count the control signals

Questions 1 and 2

	Read	Write	Steps
Fetch (the same for all)	3	4	4
Add Y: fetch + 3 steps	6	7	7
Store Z: fetch + 3 steps	6	7	7
Output: fetch + $OUT \leftarrow AC$	4	5	5
Jump Stop: fetch + $PC \leftarrow IR[11-0]$	4	5	5

Question 3

In $PC \leftarrow PC + 1$. Nothing travels on the bus: the PC increases itself. So there is a Write to the PC, but no Read.

Notice

Every step has a Write: each micro-operation changes exactly one element. Instructions that use the memory need more steps.

Read and **Write** are signals from the control unit: it switches them on in the right step. More about this in Part 3.

One bus for everything: the von Neumann bottleneck

In Exercise 3, 4 of the 7 steps of Add Y were the fetch. Instructions and data travel on the **same bus** to the **same memory**, so they must take turns.

Von Neumann (MARIE)



- + one memory: programs and data share the space (Day 1, Decision 2)
- the CPU cannot fetch an instruction and read data **at the same time**

Harvard



- + two separate roads: fetch and data **at the same time**
- two memories: “address 0 exists twice”, and the space cannot be shared

Modified Harvard (today)



- + separate roads **near** the CPU, but still one main memory
- more hardware: small copies (cache) that must be kept up to date

Today's processors are **Harvard close to the CPU** and **von Neumann further away**: the modified Harvard architecture. We will meet the L1 cache again later.

Who flips all the switches?

Day 1: the **CU** decides and coordinates, but never calculates.

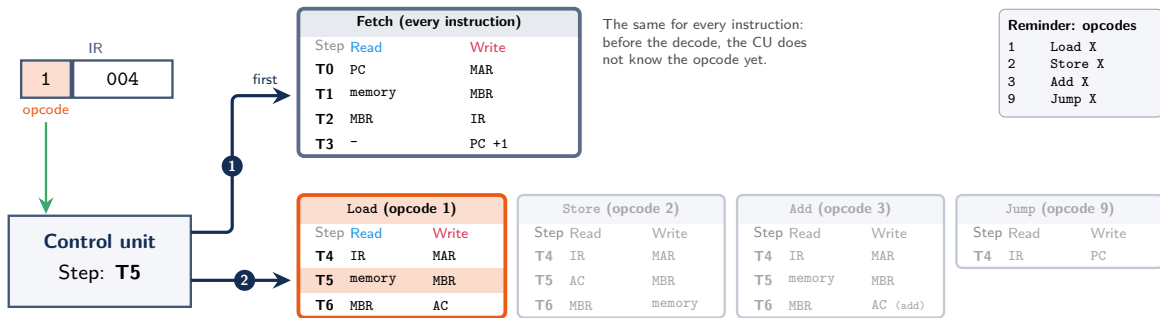
Today: an instruction is a sequence of micro-operations, one per step.



The question

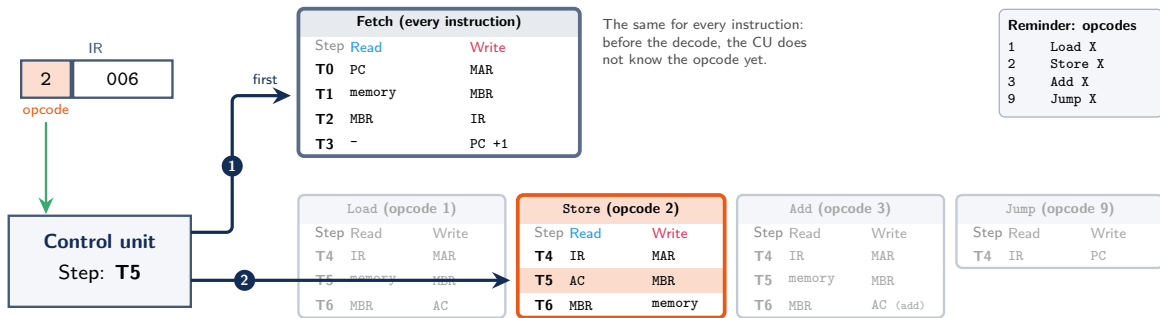
How does the CU know **what to switch on** in each step?

How the CU decides: card and row



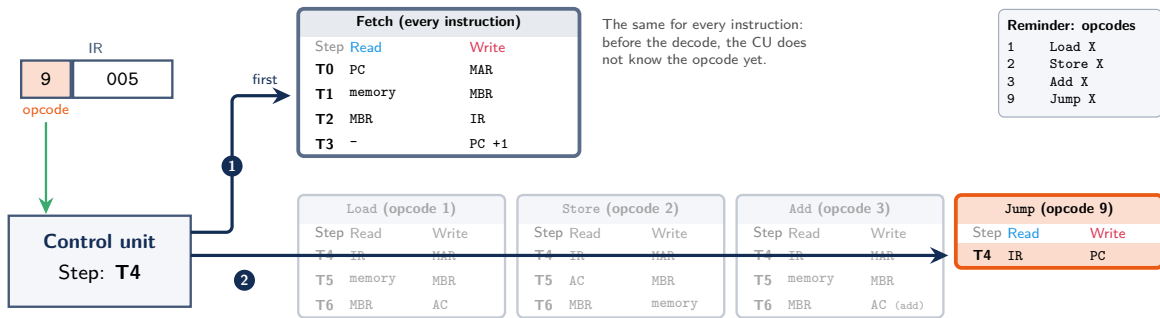
then: the opcode picks the card, the step picks the row

How the CU decides: card and row



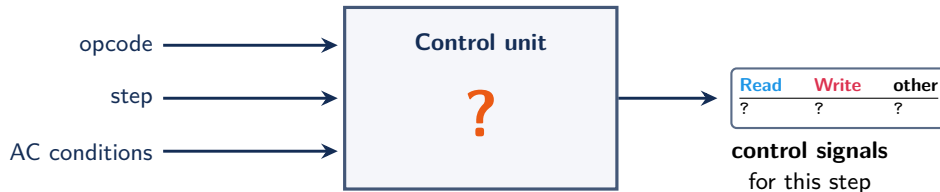
then: the opcode picks the card, the step picks the row

How the CU decides: card and row

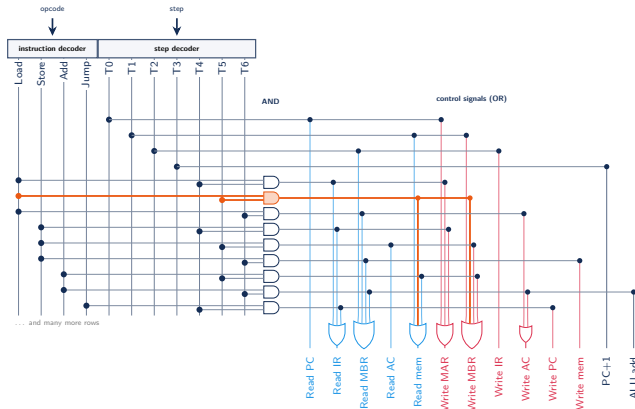


then: the opcode picks the card, the step picks the row

How can we build such a control unit?



Way 1: hardwired control unit



Decoders

one line per instruction, one line per step.

AND gates

one for every row of every card, e.g. **Load AND T5**.

OR gates

one for every control signal: it is on if *any* row needs it. The orange row switches on **Read memory** and **Write MBR**.

The cards are not stored anywhere: they are **built into the wiring**. A real CU has thousands of such gates.

Way 2: microprogrammed control unit



after the decode: jump to the start of the card;
then +1 after every step

Control memory

a small memory inside the CU that holds **all the cards**.

Microinstruction

one row: the control signals for one step.

Microprogram

one card: the rows that carry out one machine instruction.

Micro-PC

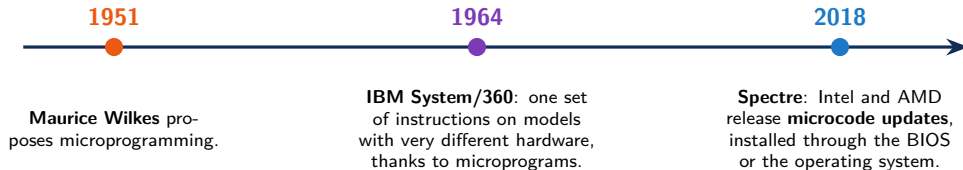
points to the current row, like the PC points to the current instruction.

Hardwired or microprogrammed?

	Hardwired	Microprogrammed
Speed	high	lower
Design	hard when there are many instructions	simpler, systematic
Changing the instructions	redesign the circuit	change the control memory
Complex instructions	awkward	natural

The best choice depends on how simple or complex the instructions are.

History and today



The processor in your laptop most likely contains microcode that can be updated.

From the control unit to two ideas: CISC and RISC

1970s

1980s

More and more powerful instructions

With a **microprogrammed** CU, a new instruction is just a few new rows in the control memory. Memory is expensive, so short programs pay off. Designers add complex instructions, e.g. one instruction that copies a whole block of memory.

CISC: Complex Instruction Set Computer
e.g. VAX, x86

Only a few simple instructions

Memory is cheaper now, and researchers (Berkeley, Stanford) notice that compilers mostly use the **simple** instructions. Idea: keep only simple instructions, all of the same kind, so a fast **hardwired** CU is enough.

RISC: Reduced Instruction Set Computer
e.g. MIPS, ARM, RISC-V

Next: what exactly is different between the two?

Two design philosophies

We will now look at **two design questions** that every processor architect must answer.

CISC

Complex Instruction Set

Different answers lead to...

VS

RISC

Reduced Instruction Set

different processor types.

→ **Question 1:** Where can an arithmetic instruction take its values from?

→ **Question 2:** Do all instructions have the same length?

Question 1: where do the operands come from?

The same statement $A = B + C$ in three styles

Accumulator (MARIE)



```
Load B
Add C
Store A
```

One register (AC). The ALU uses the AC and a value from memory.

Register-memory (x86 – CISC)



```
mov eax, [B]
add eax, [C]
mov [A], eax
```

Several registers. The ALU can use a register **and** a value from memory.

Load/store (ARM – RISC)

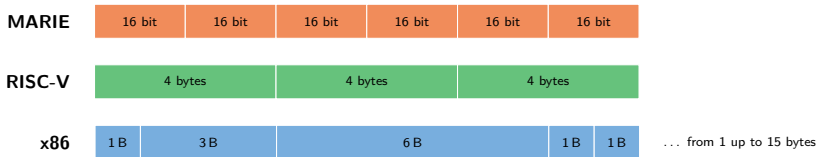


```
ldr r1, B
ldr r2, C
add r3, r1, r2
str r3, A
```

Many registers. The ALU uses **only registers**; only ldr/str touch memory.

Unlike MARIE with its single AC, modern processors (both CISC and RISC) have **many registers**.

Question 2: fixed or variable instruction length?



Fixed length

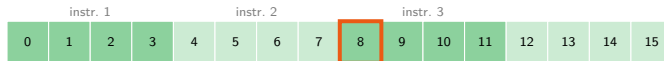
The processor knows at once where the next instruction starts, so fetching and decoding are simple.

Variable length

Common instructions can be short, so programs take less memory.

Where does the third instruction start?

“All instructions are 4 bytes”



byte 8: easy!

“Instructions are 1 to 15 bytes”



decode instruction 1, then 2,
and only then you know

With variable length, decoding is harder. We will come back to this in a future presentation with pipelining.

RISC versus CISC

	RISC	CISC
Instructions	simple, few	complex, many
Addressing modes	few	many
Instruction length	fixed	variable
Memory access	only load/store	also in arithmetic instructions
Registers	more (32 in RISC-V)	fewer (16 in x86-64)
Control unit	usually hardwired	usually microprogrammed
Code size	larger: more instructions	smaller: fewer instructions
Examples	ARM, RISC-V	x86

RISC: Reduced Instruction Set Computer

CISC: Complex Instruction Set Computer

Exercise: CISC vs RISC execution

Individual Task

Assume variables X , Y , and Z are in memory. Write the instruction sequence for $Z = (X + Y) - 2$.

1. RISC style

Available instructions:

- `load r, [mem]` (read memory into a register)
- `store r, [mem]` (write a register to memory)
- `add r, r1, r2` (add two registers)
- `sub r, r1, constant` (subtract a constant)

2. CISC style

Additional capabilities:

- `add r, [mem]` (add a memory value directly)
- `sub r, constant` (subtract a constant from a register)

Compare your two solutions:

- How many instructions did you write for RISC vs. CISC?
- How many times does each version access main memory?
- Why does the CISC version need fewer instructions?

Solution: CISC vs RISC execution

1. RISC style: 5 instructions



```
load  r1, [X]      ; r1 = X
load  r2, [Y]      ; r2 = Y
add   r3, r1, r2    ; r3 = X + Y
sub   r3, r3, 2     ; r3 = r3 - 2
store r3, [Z]      ; Z = r3
```

2. CISC style: 4 instructions



```
load  r1, [X]      ; r1 = X
add   r1, [Y]      ; r1 = r1 + Y
sub   r1, 2        ; r1 = r1 - 2
store r1, [Z]      ; Z = r1
```

Instructions

RISC: 5, CISC: 4.

Memory accesses for data

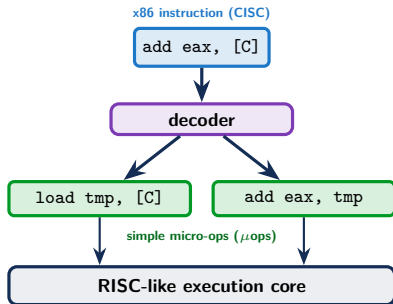
Both: 3. Read X, read Y, write Z.

Why fewer in CISC?

add r1, [Y] reads memory **and** adds in one instruction.

CISC does not save memory accesses: the load is still there, only **hidden inside** add. The work is the same, packed differently.

Today: the boundary is blurred



x86 (Intel, AMD)

CISC on the outside. Since the mid-1990s (Intel Pentium Pro, 1995) instructions are translated **inside** into simple, RISC-like μ ops.

ARM (phones, Apple M chips)

RISC instruction set, but a very complex inside.

A μ op is a small RISC-like instruction. It is not the same as a MARIE micro-operation (one step such as $MAR \leftarrow PC$), but the idea is similar: split big work into small pieces.

RISC and CISC are **design philosophies**, not rigid categories: modern processors combine ideas from both.

What we saw today

